

Risk-free Selfish Mining in Hybrid Predictability Model. A Case Study on Polkadot's NPoS

Mingfei Zhang
Shandong University
Jinan, Shandong, China
State Key Laboratory of Cryptography and Digital
Economy Security
Beijing, China
mingfei.zh@mail.sdu.edu.cn

Xinyu Lei
Tsinghua University
Beijing, China
lei-xy24@mails.tsinghua.edu.cn

Rujia Li*
Tsinghua University
State Key Laboratory of Cryptography and Digital
Economy Security
Beijing, China
rujia@tsinghua.edu.cn

Sisi Duan*[†]
Tsinghua University
State Key Laboratory of Cryptography and Digital
Economy Security
Beijing, China
duansisi@tsinghua.edu.cn

Abstract

Selfish mining attack is a well-known attack in blockchains where Byzantine validators collude to gain more revenue than their fair share. Prior studies show that selfish mining attack can be risk-free in the longest-chain variants of Proof-of-Stake (PoS) protocols, i.e., Byzantine validators will never be caught and do not take any risk of losing their revenue. However, it is still unclear whether risk-free selfish mining is possible for PoS in the non-global predictability model [EC 2019], where in the global unpredictability model, validators can predict the roles of *all* validators in advance. In this work, we study the nominated PoS protocol by Polkadot, a cryptocurrency with a market cap of over 7 billion USD (top 20 blockchains). Polkadot's PoS protocol falls into a hybrid predictability model (i.e., non-global predictability model) that has never been well studied before. We present the first two risk-free (and meanwhile profitable) selfish mining attacks against Polkadot's PoS protocol. In our attack, a mining pool with 33% Byzantine validators can launch risk-free selfish mining and earn 7.09% extra block rewards in expectation. Our experimental results using Polkadot's implementation show that our risk-free attacks achieve similar profit as conventional selfish mining attack, such as the well-known selfish mining against Bitcoin.

CCS Concepts

• Security and privacy → Distributed systems security.

*Corresponding authors.

[†]Sisi is also with National Financial Cryptography Research Center, Zhongguancun Laboratory, and Shandong Institute of Blockchains.



This work is licensed under a Creative Commons Attribution 4.0 International License. WWW '26, Dubai, United Arab Emirates.

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792560>

Keywords

Blockchain; Proof-of-Stake; Selfish mining; Polkadot

ACM Reference Format:

Mingfei Zhang, Rujia Li, Xinyu Lei, and Sisi Duan. 2026. Risk-free Selfish Mining in Hybrid Predictability Model. A Case Study on Polkadot's NPoS. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792560>

1 Introduction

Selfish mining [14], in which multiple Byzantine validators may collude and gain more revenue than their fair share, is a well-known attack against Proof-of-Work (PoW) based blockchains (e.g., Bitcoin, Ethereum 1.0 [15]) and Proof-of-Stake (PoS) based blockchains (e.g., Ethereum 2.0 [6], Polkadot [27]). Generally speaking, the strategy of the Byzantine validators is first to produce a *private branch* consisting of blocks by the Byzantine validators and then withhold the branch. When the length of the private branch is longer than the *public branch* (i.e., consisting of blocks by honest validators), the Byzantine validators release the entire private branch so the private branch now becomes the longest chain (also known as the canonical chain). As a result, the public branch will be discarded, and honest validators fail to get their mining fees.

Selfish mining can be classified into two categories depending on whether validators can predict the roles of other participants [5]: global predictability model and local predictability model. For protocols in the global predictability model, every validator knows in advance the identities of validators that are eligible to propose a block in each slot. For protocols in the local predictability model, each validator knows in advance when it will be eligible to produce a block, but does not know the roles of other validators.

Selfish mining may not be risk-free: Byzantine validators may lose block rewards if an ambitious strategy is taken. This is because it is not always possible to create a private branch longer than the public branch. For instance, if Byzantine validators continue to grow the private branch but the public branch turns out to be

longer, the private branch is discarded and the Byzantine validators lose their revenue.

It is not surprising that selfish mining is more profitable in the global predictability model than in the local predictability model. Brown-Cohen, Narayanan, Psomas, and Weinberg (BNPW) [5] have shown that selfish mining in the global predictability model can be risk-free, where the Byzantine validator does not suffer from the risk of losing its revenue (see other selfish mining attacks in Appendix A). Indeed, as the roles of all validators can be predicted, the Byzantine validators can know exactly whether the length of the private branch will be longer than the public branch and when the private branch should be released. Risk-free selfish mining is attractive for risk-averse attackers [16, 24] who do not want to suffer from any revenue loss even in extreme cases.

Our research problem. Polkadot [27], a leading blockchain platform, adopts a hybrid predictability model. Specifically, in each *slot* (i.e., the smallest time unit in which some validators propose their blocks), two types of validators are selected: *primary* leaders, and *secondary* leaders. The selection of the primary leaders follows the local predictability model, and the selection of the secondary leaders follows the global predictability model. Both primary leaders and secondary leaders can propose new blocks, but at most one block can be included in the longest chain. Such a hybrid design raises our research question:

Does there exist a risk-free selfish mining attack in the hybrid predictability model?

Our approach. In this paper, we answer this question by showing that selfish mining can be risk-free in the hybrid predictability model while being *profitable*, i.e., the attack can gain the same level of profit compared to other known selfish mining attacks. We present two attacks. Our first attack is rather straightforward: by only looking at the selection of the primary leaders, Byzantine validators launch a selfish mining attack only when they are certain that the private branch will be longer than the public branch. This attack exploits the observation that although Byzantine validators cannot predict the roles of honest validators, they know their own roles in advance. Such an attack, while being risk-free, is not that profitable. Specifically, only 2% higher rewards is expected when Byzantine validators control 33% stake.

We further expand our attack strategy to a more profitable attack that is also risk-free. Our second attack exploits the implementation of how validators handle block proposals *in the future* (i.e., proposals with higher slot numbers). We show that by deliberately proposing and releasing the blocks, Byzantine validators can create a longer private branch than our first attack and thus gain more profit. Using our new attack strategy, 33% Byzantine validators can obtain more than 7% expected income than their fair share.

We can merge both attacks into one attack to gain more profit. We summarize the profit of our attack by varying the stake owned by Byzantine validators, and compare our attack with known selfish mining attack to Bitcoin [14] and Ethereum [20] (see attack strategies in Appendix B).

Responsible disclosure. For ethical reasons, we have discussed our findings with Polkadot Community Foundation, and the development team has taken measures since then to mitigate the attack.

Our contributions We summarize our contributions below.

stake (%)	our work	Bitcoin [14]	Ethereum [20]
5	1.24	0.24	0.05
10	2.41	0.92	0.19
15	3.45	2.01	0.42
20	4.36	3.51	0.96
25	5.28	5.49	1.60
33	7.07	9.71	3.42

Table 1: Comparison of the profit of selfish mining attacks by varying the fraction of stake owned by Byzantine validators.

- We present a formal treatment of the Polkadot consensus protocol. While the official documentation provides only informal descriptions, our work introduces precise models, notations, and definitions that capture its core mechanisms of Polkadot consensus (Section 2).
- We provide two risk-free selfish mining attacks on the Polkadot consensus. We show an instantiation of our attack in the PoS protocol used by Polkadot. Our analysis shows that Byzantine validators can obtain 7% expected extra payout than their fair share, without taking any risks of losing any revenue (Section 3 & Section 4).
- We implement the attacks. We run the experiments with 297 validators (matching the setup of the current production system), and each experiment lasts for five epochs (i.e., 20 hours). Our experiment results match our theoretical analysis (Section 5).

2 Review of the Polkadot Consensus Mechanism

The consensus mechanism of Polkadot is called Nominated Proof-of-Stake (NPoS) [8]. Specifically, each party in the system nominates some validators by locking its stake. Based on the total amount of stake locked by nominators, a set of validators is elected. These validators then run a PoS protocol to reach an agreement on the order of blocks.

In this section, we review the Polkadot PoS protocol, assuming that the set of validators has already been nominated. We begin by presenting the system model and notations relevant to our attacks, followed by providing a brief review of the workflow of Polkadot PoS. Our presentation largely follows the official document of Polkadot [2] and the source code [1].

2.1 System Model and Threat Model

A set of n validators $\{v_1, \dots, v_n\}$ run the PoS protocol. The adversary may corrupt up to f validators where $f < n/3$. Corrupted validators are called Byzantine validators. Non-corrupted validators are called honest validators. Without loss of generality, we assume that n is a static number. Polkadot assumes a partially synchronous network model [13], i.e., there exists an unknown upper bound Δ for message transmission and processing delay. Our selfish mining attack can still be launched even in a synchronous network model, where the value of Δ is known. This is the *weakest* assumption one can make on any attack.

Parameter	Meaning
n	the number of validators
f	the number of Byzantine validators
t	a slot number
C	a chain received by some validators
$C_{t,t+1}^{adv}$	the private chain by the adversary from slot t to slot $t + 1$
$C_{t,t+1}^{hon}$	the public chain by the adversary from slot t to slot $t + 1$
d	the confirmation depth
	a finalized block
	a primary block
	a secondary block
	a block in discarded branch
	a block from a Byzantine validator

Table 2: Notations.

2.2 Notations

The PoS protocol of Polkadot is a combination of Blind Assignment for Blockchain Extension (BABE) [3] and GHOST-based Recursive Ancestor Deriving Prefix Agreement (GRANDPA) [26]. We summarize our notations in Table 2.

Epoch and slot. Time is divided into epochs, and each epoch consists of 2,400 slots. In the current production system, each slot lasts 6 seconds. That is, each epoch lasts 4 hours.

Block and leader. We use b to denote a block. Each block consists of the slot number, the hash of the parent block, and a batch of transactions. The first block in the system is called the *genesis block*. Every validator maintains a block tree $\mathcal{T}$ to store the blocks it has received. We use C to denote a chain of blocks. The chain C seen by each validator is part of its block tree, and the longest chain in $\mathcal{T}$ is called the *canonical chain*. The leaf node of the canonical chain is called the *head* of the chain. Blocks are identified as *primary blocks* and *secondary blocks*, which are proposed by *primary leaders* and *secondary leaders*, respectively.

The system allows a validator to accept messages *from the past or future*. In particular, if a validator v_i receives a block containing a timestamp that is either later or earlier than v_i 's local clock, v_i still considers the message as valid and processes the block. Notably, our second attack exploits a special rule:

▷ *Block acceptance rule exploited by this work.* If a validator is in slot t and it receives a block with slot t' , the block is considered valid if $t' - t \leq 1$.

Fork choice. BABE allows validators to continue to propose new blocks according to the *longest-chain rule*. The longest-chain rule is also called the *fork-choice rule*, denoted as ForkChoice. Each validator uses the output of the ForkChoice function to determine its canonical chain. A validator will only extend its canonical chain when creating a new block. Given the block tree, the fork choice works as follows (an example is given in Fig. 1):

1. Choose the set of chains Set_1 that extend the last finalized block from the block tree.
2. Choose the set of chains Set_2 with the most primary blocks from Set_1 .

3. Output the chain with the most secondary blocks from Set_2 . Break the tie by selecting the first received chain.

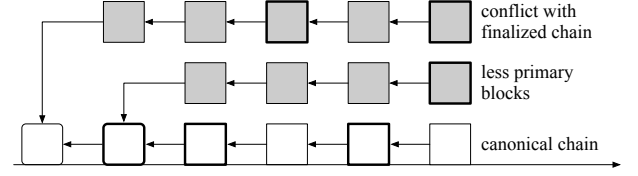


Figure 1: Block production mechanism and the longest chain rule in BABE.

Finality. GRANDPA is the finality gadget that finalizes the proposed blocks. After a block is finalized, the order of the block will never be reversed. In the failure-free case, consider the canonical chain c , where the head is block b and the height (i.e., the number of blocks on c between the genesis block and b) of b is h , GRANDPA finalizes the block with height $h - d$, where d is the confirmation depth. d is not a fixed value; it changes due to the network condition. According to SubScan¹, the average value of d is five. Our attack does not care about the finalization rules, and we can simply assume that a block b is finalized after seven more blocks are proposed.

Workflow. We present the workflow of Polkadot PoS protocol in Fig. 2. Each validator v_i first identifies its role at the beginning of each slot. If v_i is the primary leader, v_i first uses the ForkChoice function to obtain the *head* of the canonical chain (line 03). It then creates a new primary block $b = (\text{PRIMARY}, t, v_i, H(\text{head}), \text{txs})$ by extending the canonical chain, where t is the slot number, head is the parent block of b , and txs is a batch of transactions (line 05). It then sends b to all validators. Otherwise, if v_i is a secondary leader, it also creates a secondary block $b = (\text{SECONDARY}, t, v_i, H(\text{head}), \text{txs})$ and sends b to all validators (lines 11-13).

Upon receiving a valid block b' from validator v_j , validator v_i adds b' to its block tree $\mathcal{T}$ (lines 14-16). One particular rule exploited by our attack is the *block acceptance rule* mentioned in Section 2.2.

2.3 Primary and Secondary Leadership

In each slot, two types of validators are selected, and only selected validators are eligible to create a block proposal: *primary leaders* and *secondary leaders*. The blocks proposed by the primary leaders and the secondary leaders are called the *primary blocks* and the *secondary blocks*, respectively. The selection of primary leaders and secondary leaders belongs to the local predictability model and global predictability model, respectively. We thus call the leader selection rule of Polkadot the hybrid predictability model. Specifically, the leader selection rules are summarized below.

▷ *Primary leadership.* The selection of the primary leader uses Verifiable Random Functions (VRF) [19]. In particular, at the beginning of every slot t , every validator v_i creates a VRF evaluation using the slot number and *epoch randomness seed* (computed based on the VRF evaluations of prior epochs, which is unpredictable until the beginning of the current epoch). If v_i 's VRF value is lower than a pre-specified threshold, v_i becomes the primary leader and is

¹SubScan: <https://polkadot.subscan.io/>

The Polkadot PoS protocol for validator v_i

local parameters: slot t , block tree $\mathcal{T}$

```

01 upon slot  $t$  do
02   as primary leader
03     let  $head$  be the output of ForkChoice
04     let  $txs$  be the patch of transactions
05     build block  $b = (\text{PRIMARY}, t, v_i, H(head), txs)$ 
06     if the slot of  $head$  is smaller than  $t$  then
07       then send  $b$  to all validators
08   as secondary leader
09     let  $head$  be the output of Forkchoice
10     let  $txs$  be the patch of transactions
11     build block  $b = (\text{SECONDARY}, t, v_i, H(head), txs)$ 
12     if the slot of  $head$  is smaller than  $t$  then
13       send  $b$  to all validators
14 upon receiving block  $b'$  from validator  $v_j$ 
15   if the slot of  $b'$  is at most one slot earlier than  $t$ 
16     then  $\mathcal{T} \leftarrow \mathcal{T} \cup \{b'\}$ 

```

Figure 2: The Polkadot PoS protocol. Codes are shown for validator v_i . $H()$ denotes the hash function.

eligible to create a block b . When v_i sends b_j to the validators, it also includes its VRF, so other validators can verify the role of v_i . As the VRF generates pseudorandom values and validators do not reveal their identities before sending the blocks, the selection of the primary leadership is pseudorandom and unpredictable.

It is possible that none of the validators is selected or multiple validators are selected. According to Polkadot's current configuration, a *BABE Constant* c is used to denote the probability that there is at least one primary leader in a slot. Currently, $c = 1/4$. Thus, around three out of four slots do not have a primary leader.

▷ *Secondary leadership.* To select the secondary leader at the beginning of an epoch e , each validator v_i first calculates the epoch randomness seed of epoch $e - 1$. It then uses the seed to calculate the secondary validator in each slot. Unlike the primary leadership, there is only one secondary leader in each slot. Additionally, the secondary leader of each slot can be learned by *all* validators at the beginning of an epoch (to be concrete, after the epoch randomness seed is generated).

3 Risk-Free Selfish Mining via Withholding

In this section, we present our risk-free selfish mining attack via withholding. We assume that the adversary already controls a fraction of validators. For now, we do not discuss the concrete fraction and focus on how the attack can be launched. Later, we show that the effect of our selfish mining attacks roughly matches that of known selfish mining attacks: as the fraction of validators controlled by the adversary grows, the additional reward of the adversary also grows.

Briefly speaking, our attack exploits the fact that although the roles of honest primary validators are not predictable until they reveal their identities, Byzantine validators can predict their roles at the beginning of each epoch. Thus, Byzantine validators can

construct the private branch according to the partial information and ensure that the attack is risk-free.

3.1 The Attack via Withholding

In a nutshell, our attack requires Byzantine validators to withhold their blocks (and form a private branch) and release them at the *right time*. The crucial problem is *when to start withholding and when to release* the private branch. The core challenge lies in Polkadot's hybrid predictability model. As mentioned above, Byzantine validators can predict their own roles and obtain partial information. Specifically, below is a summary of what Byzantine validators know at the beginning of an epoch e :

- They know all the secondary leaders in epoch e , as the selection of secondary leadership follows the global predictability model (Section 2.3).
- They know the slots in which the primary leaders are Byzantine. Let these slots be S_a . For other slots, Byzantine validators do not know whether there exist a primary leader or not. This is mainly because the selection of primary leadership follows the local predictability model, and the probability that there exists a primary leader is $c = 1/4$. Let the slots in which Byzantine validators will not be primary leaders be S_h .

The key insight of our attack is based on the fork choice rule that favors primary blocks over secondary blocks (Section 2.2). To make sure our attack is risk-free, the Byzantine validators must guarantee that their private branch has *at least the same number of primary blocks and secondary blocks* as the public branch. Namely, if the public branch has more blocks, the public branch might not be discarded and our attack is not risk-free.

Therefore, we always consider the worst case, i.e., before the beginning of every slot $t \in S_h$, we always assume that there exists an honest primary leader. Based on such an observation, we present our attack in Fig. 3.

Discussion. Note that although our initial condition requires at least one primary leader to be Byzantine, our attack is continued when two consecutive primary leaders are Byzantine. To see why this is the case, consider that slot t satisfies $t \in S_a$ and $t + 1 \notin S_a$. When slot t begins, the termination condition is already satisfied. This is because rule (2) assumes that there exists a virtual block in slot $t + 1$. Thus, the primary block is released before slot t begins, i.e., the attack is stopped once it is started.

We would like to emphasize that we intentionally design our attack like this to make our attack risk-free. Indeed, although the probability of selecting a primary leader in each slot is $c = 1/4$, with certain probability, there exist *multiple* primary leaders. We thus assume the worst case. To do so, when we construct the virtual public branch, we assume that there always exists an honest primary leader for *every* slot. While the protocol proceeds, validators continue to receive the blocks by honest validators, and can then fill in the public branch with *real* blocks, according to rule (1).

Impact of confirmation depth. The confirmation depth d is a tunable value in Polkadot. Our attack strategy is robust to changes in confirmation depth. Specifically, the Byzantine validators continuously monitor the finalization status and decide when to release the private branch. The Byzantine validator will then release the private branch before any conflicting block becomes finalized.

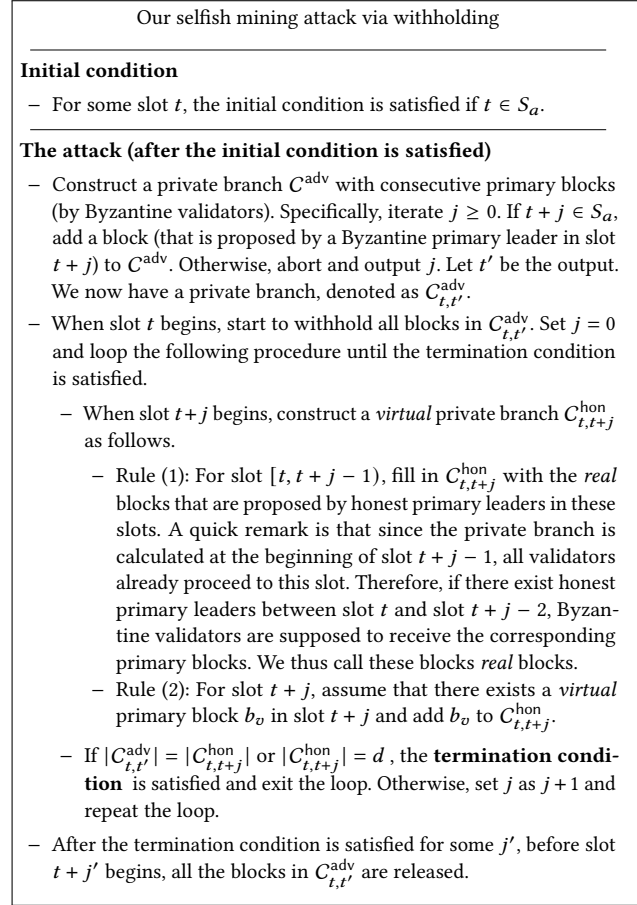


Figure 3: Our risk-free selfish mining attack via withholding.

Revenue Analysis. We model the block reward of risk-free selfish mining via withholding as a Markov Decision Process (MDP) [25]. Our theoretical analysis shows that 33% of validators can obtain approximately 2% additional block rewards compared to their fair share. The detailed MDP formulation and state transition probabilities are provided in Appendix C.

3.2 Attack Scenarios

Below, we show two scenarios for the attack.

Scenario 1. As shown in Fig. 4, $(t + 1, t + 2) \in S_a$. For slot $t + 1$, the initial condition is satisfied since $t + 1 \in S_a$. Since $t + 2 \in S_a$, the virtual private branch is $C_{t+1,t+2}^{\text{adv}} = \{b_a, b'_a\}$. The attack can be started when slot $t + 1$ begins.

- (1) **Step 1 (Fig. 4b):** When slot $t + 1$ begins, both b_a and b'_a are withheld. Here, the virtual public branch is assumed to be $C_{t+1,t+1}^{\text{hon}} = b_v$, where b_v is a virtual block;
- (2) **Step 2 (Fig. 4c):** In slot $t + 1$, there exists an honest primary leader that produces block b_h . The virtual public branch $C_{t+1,t+1}^{\text{hon}}$ now becomes a *real* branch. We use $C_{t+1,t+1}^{\text{hon}} = b_h$ to denote it. Before slot $t + 2$ begins, Byzantine validators repeat the calculation of virtual branches, and evaluate the termination condition. By our attack, a primary virtual block in slot $t + 2$ is assumed. Thus,

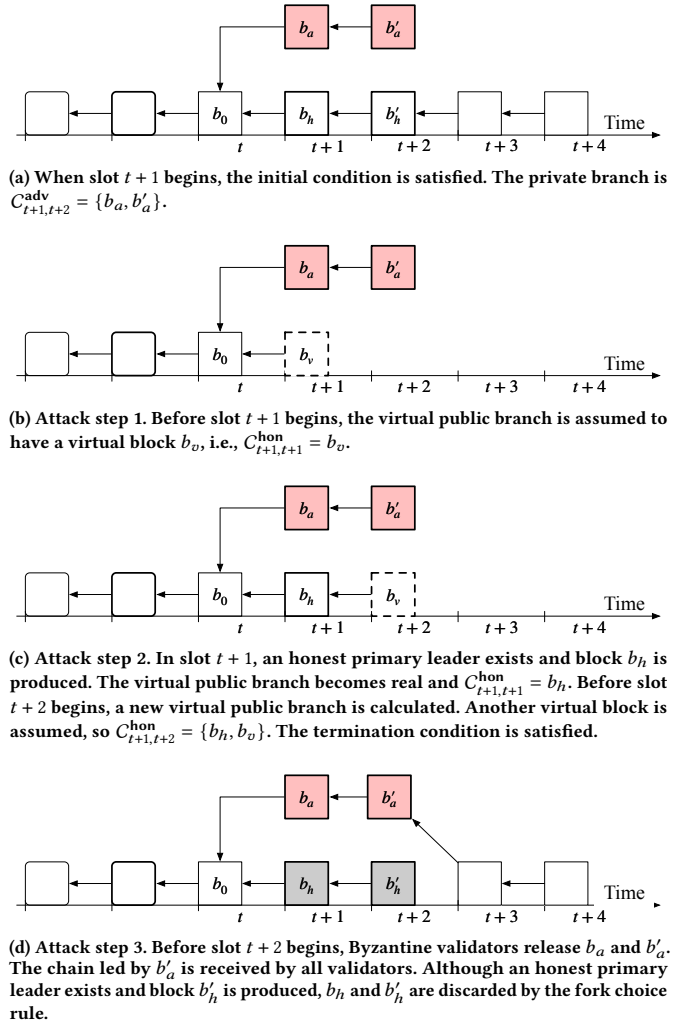


Figure 4: Scenario 1 of our risk-free selfish mining attack via withholding.

$C_{t+1,t+2}^{\text{hon}} = b_h, b_v$, where b_v is a virtual block. Since $|C_{t+1,t+2}^{\text{adv}}| = |C_{t+1,t+2}^{\text{hon}}|$, the termination condition is satisfied.

- (3) **Step 3 (Fig. 4d):** Before slot $t + 2$ begins, Byzantine validators immediately release their private branch. There indeed exists an honest primary leader that creates b'_h . However, since the private branch is released before slot $t + 2$ begins, b_a and b'_a are expected to be received earlier than the branch led by b'_h . According to the fork choice rule, when two branches have the same number of primary blocks, the fork choice break the tie by selecting the first received chain. The branch led by b'_a becomes the canonical chain, and b_h and b'_h are discarded.

Scenario 2. As shown in Fig. 5, we also have $(t + 1, t + 2) \in S_a$. Similar to scenario 1, the attack can be started when slot $t + 1$ begins and $C_{t+1,t+2}^{\text{adv}} = \{b_a, b'_a\}$.

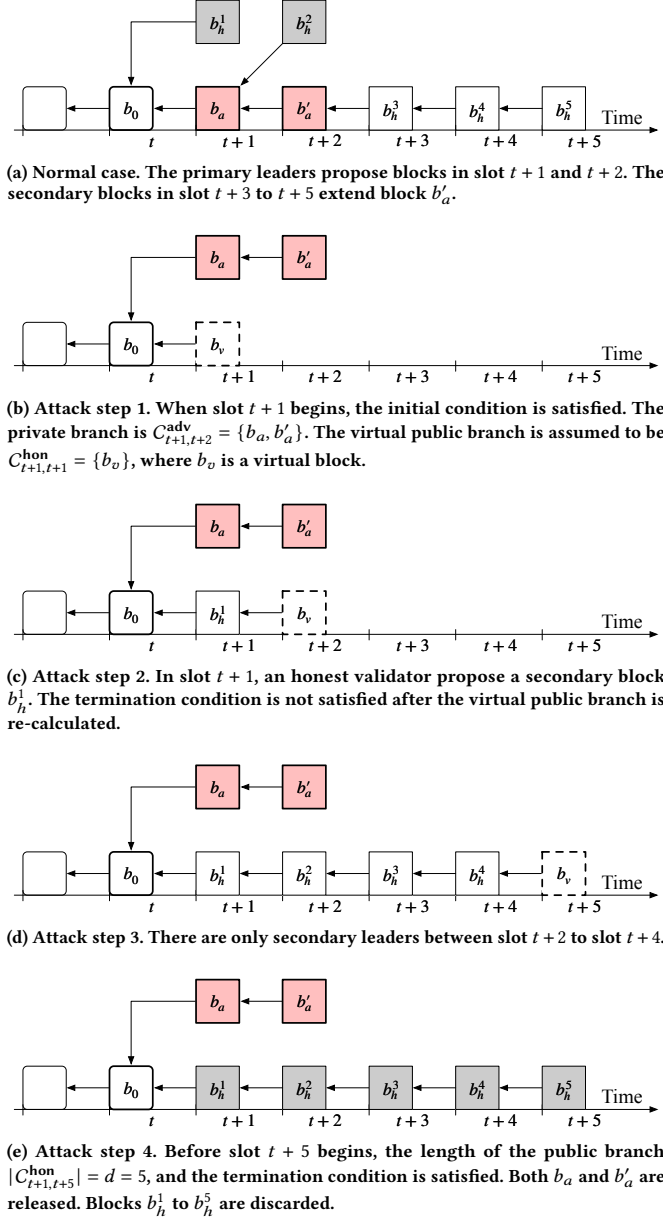


Figure 5: Scenario 2 of our selfish mining attack via withholding.

- (1) **Step 1 (Fig. 5b):** When slot $t+1$ begins, Byzantine validators withhold block b_a and b'_a . Also, construct the virtual private branch is $C_{t+1,t+1}^{\text{hon}} = \{b_v\}$, where b_v is a virtual block.
- (2) **Step 2 (Fig. 5c):** In slot $t+1$, an honest validator proposes a secondary block b_h^1 . The public branch now becomes $C_{t+1,t+1}^{\text{hon}} = \{b_h^1\}$. Before slot $t+2$, by constructing the virtual public branch as $C_{t+1,t+2}^{\text{hon}} = \{b_h^1, b_v\}$, the termination condition is not satisfied since $|C_{t+1,t+2}^{\text{adv}}| > |C_{t+1,t+2}^{\text{hon}}|$.

- (3) **Step 3 (Fig. 5d):** During slots $t+2$ to $t+4$, there is no honest primary block. Thus, the termination condition is still not satisfied.
- (4) **Step 4 (Fig. 5e):** Before slot $t+5$, the length of the public branch $|C_{t+1,t+5}^{\text{hon}}| = d = 5$. The termination condition is satisfied. Byzantine validators release blocks b_a and b'_a . The chain led by b'_a becomes the canonical chain and blocks b_h^1 to b_h^5 are discarded.

4 Risk-Free Selfish Mining via Early Release

In this section, we present our second risk-free selfish mining attack. We call this attack selfish mining via early release. Recall that the block acceptance rule (defined in Section 2.2) allows each validator to accept a block proposed in one slot ahead. Our attack exploits this rule by creating a block *ahead of time*. Using this new attack, an adversary that controls 33% of the stake can gain approximately 5.09% additional reward than the fair share.

4.1 The Attack

We illustrate our attack in Fig. 7. Consider slot $t+1$ in which a malicious validator v_a is a secondary leader in slot $t+1$ and the primary leader v_h is honest. The goal of our attack is to ensure that the block proposed by v_h will *never* be included in the canonical chain of any party. For simplicity, let b_0 be the block proposed in slot t and let the chain led by b_0 be the canonical chain of all honest validators. We show our attack in Fig. 6.

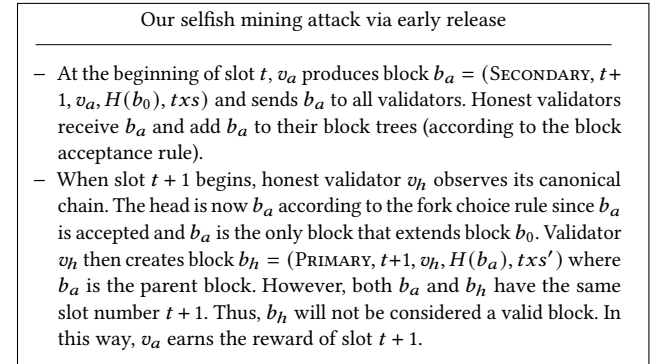


Figure 6: Our risk-free selfish mining attack via early release.

Risk analysis. The attack is risk-free. If no honest primary leader exists in slot $t+1$, block b_a is still valid, i.e., the reward received by v_a is *fair*. If an honest primary leader v_h exists, the reward that is supposed to be earned by v_h is now earned by v_a .

Revenue Analysis. The adversary can launch this attack whenever it controls a secondary leader in any slot. Let p_{adv} denote the fraction of slots in which the adversary controls the secondary leaders. For f malicious validators among n total validators, $p_{\text{adv}} = f/n$ by the leader selection rules.

Consider a slot in which the adversary launches the attack. Let p_p^h denote the probability that at least one honest validator serves as the primary leader in a slot. The expected additional payout of the attack is:

$$R_{\text{avr}} = p_{\text{adv}} \cdot p_p^h \cdot (1 - c \cdot f/n) \cdot R_p,$$

where R_p represents the block reward.

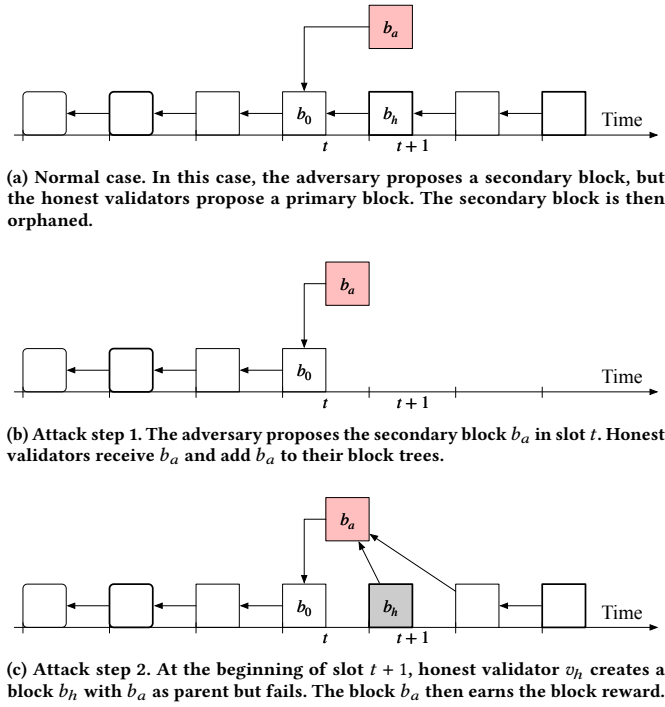


Figure 7: Our second risk-free selfish mining attack.

Here, $(1 - c \cdot f/n)$ is the probability that no Byzantine validators are selected as primary leaders. Since these events are independent, we can multiply the probabilities. Using Polkadot's current parameters ($c = 1/4$) and $f/n = 0.33$, the adversary gains approximately 5.09% additional block rewards.

We can merge both attacks into one attack, so the adversary gains approximately 7.09% expected additional block rewards

5 Implementation and Evaluation

We implement our risk-free selfish mining attacks on a local Polkadot testnet to validate the theoretical analysis and quantify the practical impact of our attack strategies².

5.1 Implementation

We implement our risk-free selfish mining attacks using the Polkadot SDK³, the official Rust-based implementation framework for Polkadot relay chain. We utilize the latest version of the SDK at the time of implementation, which provides the complete BABE consensus mechanism and GRANDPA finality gadget as described in Section 2.

We implement our attacks using the Polkadot SDK's consensus layer by modifying the block production and network communication modules. In particular, we modify the protocol in the following two aspects:

Attack via withholding blocks. Our first attack is implemented by modifying the block authoring pipeline in the BABE consensus

engine. We set up f validators that will apply the attack strategy whenever possible. All controlled validators apply the initial condition described in Section 3 by monitoring their assigned primary leadership slots for each epoch. When the primary leaders of two consecutive slots are controlled by the adversary, the blocks are withheld. Then the controlled validators track the public branch to determine the optimal release timing. Once the termination condition is satisfied, the withheld blocks are released immediately to all validators.

Attack via early release. Our second attack requires the Byzantine validators to produce blocks earlier than expected. To achieve this, we encapsulate all the time retrieval interfaces of the Byzantine validators and redirect them to a fake API. In our implementation of the fake API, the adversary monitors the secondary leadership slots. In each slot t , if the secondary leader in slot $t + 1$ is controlled by the adversary, the block is created immediately and sent to all validators.

5.2 Experimental Setup

We configure a local testnet to evaluate the attacks. The testnet operates with up to 100 validators. We believe an assessment in a local network is sufficient to assess our attack. Namely, we would like to validate that our attack can be successfully launched. As emphasized earlier in the paper, since our attack can be launched when the network is synchronous, it can also be easily launched under other network conditions.

We vary the fraction of validators controlled by the adversary (denoted as f), and evaluate six distinct configurations: $f = 5, 10, 15, 20, 25$, and 33 . Each experiment lasts for five epochs (2,400 slots per epoch). We initialize the testnet with identical stake distributions.

5.3 Evaluation Results

We conduct experiments to evaluate our risk-free selfish mining attacks. Our evaluation focuses on the additional reward earned by Byzantine validators beyond their fair share. We summarize our results in Fig. 8.

The experimental results roughly match our theoretical analysis: When $f = 33$, the adversary earns 7.07% more block rewards than their fair share, closely aligning with our theoretical prediction of approximately 7.09% additional revenue. All of our experiments show that as the fraction of validators controlled by the adversary grows, the additional block rewards of the adversary also grows.

Comparative analysis with selfish mining of Bitcoin. We also compare our selfish mining attacks with the established selfish mining attack on Bitcoin [14]. The comparison reveals that our risk-free attacks on Polkadot achieve more profit than selfish mining in Bitcoin PoW when the stake controlled by Byzantium validators is less than 20%.

6 Discussion

6.1 Countermeasure

Mitigation of the attack via early release. The attack via early release reveals a critical issue with the implementation of message processing under an asynchronous network or when the clocks of validators are not fully synchronized: if a validator receives a message *in the future*, it should not process the message immediately. First, we would like to argue that the block acceptance rule is a

²Our implementation is publicly accessible at <https://zenodo.org/records/17254647>.

³Polkadot SDK: <https://github.com/paritytech/polkadot-sdk>

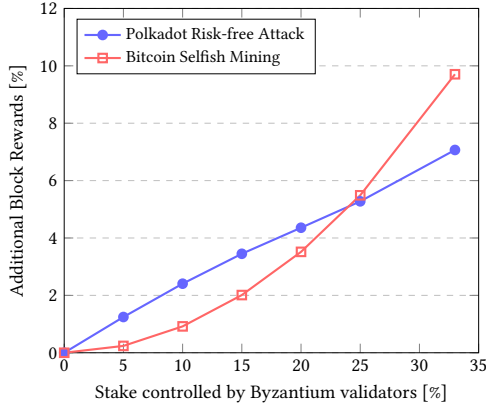


Figure 8: Additional block reward percentage vs. adversarial fraction under our risk-free selfish mining and Bitcoin selfish mining.

reasonable implementation. Namely, since the clocks of all validators are not fully synchronized (under the assumption of Polkadot), it is possible that an honest validator in slot t receives a block b with slot $t + 1$, and b is proposed by an honest leader. Second, we believe the root cause of the issue is that the block is processed at the *wrong* time. Namely, the following countermeasure can mitigate the attack.

- If a validator p_i is in slot t and it receives a block b with slot t' such that $t' > t$, it stores b in its cache.
- Whenever p_i enters slot t'' , it extracts all the cached blocks with slot t' and processes the blocks according to the specification of the protocol. Namely, block b will be processed by p_i only when b_i enters slot t' .

To see why the above solution can prevent our attack via early release, consider the scenario shown in Fig. 7. Upon receiving b_a in slot t , validators cache b_a since they have not entered slot $t + 1$ yet. Thus, the honest primary leader will propose b_h by setting b_0 as its parent block. After honest validators enter slot $t + 1$, they will process b_a and add b_a to their block trees. Now, by the end of slot $t + 1$ (when the network is synchronous), all validators will receive b_h . Since b_h is proposed by a primary leader, the chain led by b_h is the canonical chain by the fork choice rule. The selfish mining attack will not be successful.

Mitigation of selfish mining attack in general. Selfish mining attack is one kind of *reorganization attack*, where blocks proposed by honest validators are not included in the longest chain. Several attempts have been made to study consensus protocols that are *reorg-resilient* [9, 10, 30] for PoS protocols. The main argument is that for slot-based PoS protocols and when the network is asynchronous, it is extremely challenging, if not impossible, to build a reorg-resilient protocol. All the solutions proposed so far are reorg-resilient when the network is synchronous, and the finality gadget needs to be adapted to prevent reorganization attacks. Meanwhile, all known solutions are inspired by conventional Byzantine fault-tolerant (BFT) protocols [4, 7, 11, 12, 28] since BFT protocols usually do not suffer from reorganization attacks. For instance, a recent work [30] uses the concept of *weak quorum certificate* [7, 12, 17] of

$f + 1$ matching votes to improve the PoS protocol used in Ethereum 2.0. By building a solution called *available attestation* via weak quorum certification, the Ethereum PoS protocol is reorg-resilient when the network is synchronous. We believe a similar approach can be adapted to improve Polkadot's PoS protocol.

6.2 Feasibility of Our Attack in Practice

To validate the feasibility of launching our attacks in practice, we use the empirical measurements by Polkadot mainnet. Specifically, we collect the data from the mainnet for the period of June to September 2025. The dataset contains over 300,000 blocks. Our main analysis results include: 1) about 25.2% of the finalized blocks are primary blocks, roughly matching $c = 1/4$; 2) the average time each validator receives a proposed block is 0.6 seconds, i.e., the time each validator receives the block minus the beginning time of the slot. Thus, to launch our attack, Byzantine validators can release their blocks 0.6s ahead of time, so our attack can succeed. We provide details of the analysis in Appendix D.

7 Conclusion

In this paper, we demonstrate two attacks on Polkadot's Nominated Proof-of-Stake, showing that selfish mining can be both risk-free and profitable under the hybrid predictability model. The first attack exploits the selection of primary leaders and yields only 2% higher rewards with 33% stake. The second attack leverages the handling of future block proposals, enabling Byzantine validators to maintain a longer private branch and earn over 7.09% additional income with the same stake fraction.

Acknowledgment

We thank Yichen Gong for the discussion of an earlier version of the paper. This work was supported in part by the National Key R&D Program of China under 2022YFB2702800, the Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China under JYB2025XDXM114, the National Natural Science Foundation of China under 62502266 and 125B2031, China Postdoctoral Science Foundation under 2023M741949, and Tsinghua Shuimu Scholar.

References

- [1] 2025. Parity Polkadot Blockchain SDK. <https://github.com/paritytech/polkadot-sdk>. Accessed: 2025-09-25.
- [2] 2025. *Polkadot Protocol Specification*. Technical Specification. Web3 Foundation, <https://spec.polkadot.network/id-polkadot-protocol>. <https://spec.polkadot.network/id-polkadot-protocol>
- [3] Handan Kilinc Alper. 2023. BABA. <https://research.web3.foundation/Polkadot/protocols/block-production/Babe>.
- [4] Christian Berger, Hans P Reiser, and Alysson Bessani. 2021. Making reads in BFT state machine replication fast, linearizable, and live. In *SRDS*.
- [5] Jonah Brown-Cohen, Arvind Narayanan, Alexandros Psomas, and S Matthew Weinberg. 2019. Formal barriers to longest-chain proof-of-stake protocols. In *EC*.
- [6] Vitalik Buterin and Virgil Griffith. 2017. Casper the friendly finality gadget. *arXiv preprint arXiv:1710.09437* (2017).
- [7] Miguel Castro and Barbara Liskov. 2002. Practical Byzantine fault tolerance and proactive recovery. *TOCS* 20, 4 (2002), 398–461.
- [8] Alfonso Cevallos and Alistair Stewart. 2021. A verifiably secure and proportional committee election rule. In *AFT*, Foteini Baldimtsi and Tim Roughgarden (Eds.). ACM, 29–42.
- [9] Francesco D'Amato, Joachim Neu, Ertem Nusret Tas, and David Tse. 2024. Goldfish: No More Attacks on Ethereum?!. In *FC*.

- [10] Francesco D'Amato and Luca Zanolini. 2023. Recent latest message driven ghost: Balancing dynamic availability with asynchrony resilience. *arXiv preprint arXiv:2302.11326* (2023).
- [11] Sisi Duan and Haibin Zhang. 2022. Foundations of Dynamic BFT. In *SP*. 1317–1334.
- [12] Sisi Duan, Haibin Zhang, Xiao Sui, Baohan Huang, Changchun Mu, Gang Di, and Xiaoyun Wang. 2024. Dashing and Star: Byzantine Fault Tolerance from Weak Certificates. In *Eurosys*.
- [13] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. 1988. Consensus in the presence of partial synchrony. *Journal of the ACM (JACM)* 35, 2 (1988), 288–323.
- [14] Ittay Eyal and Emin Gün Sirer. 2018. Majority is not enough: Bitcoin mining is vulnerable. *Commun. ACM* 61, 7 (2018), 95–102.
- [15] Chen Feng and Jianyu Niu. 2019. Selfish mining in ethereum. In *International Conference on Distributed Computing Systems (ICDCS)*.
- [16] Erick Galinkin, John Carter, and Spiros Mancoridis. 2021. Evaluating attacker risk behavior in an internet of things ecosystem. In *International Conference on Decision and Game Theory for Security*. Springer, 354–364.
- [17] Neil Girdharan, Florian Suri-Payer, Ittai Abraham, Lorenzo Alvisi, and Natacha Crooks. 2024. Autobahn : Seamless high speed BFT. In *SOSP*.
- [18] Rujia Li, Mingfei Zhang, Xueqian Lu, Wenbo Xu, Ying Yan, and Sisi Duan. 2025. BunnyFinder: Finding Incentive Flaws for Ethereum Consensus. *Cryptology ePrint Archive* (2025).
- [19] Silvio Micali, Michael O. Rabin, and Salil P. Vadhan. 1999. Verifiable Random Functions. In *FOCS*. 120–130.
- [20] Ábel Nagy, János Tapolcai, István András Seres, and Bence Ladóczki. 2025. Forking the RANDAO: Manipulating Ethereum's Distributed Randomness Beacon. In *CCS*.
- [21] Kartik Nayak, Srikanth Kumar, Andrew Miller, and Elaine Shi. 2016. Stubborn mining: Generalizing selfish mining and combining with an eclipse attack. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 305–320.
- [22] Michael Neuder, Daniel J Moroz, Rithvik Rao, and David C Parkes. 2021. Selfish behavior in the tezos proof-of-stake protocol. *arXiv preprint arXiv:1912.02954* (2021).
- [23] Michael Neuder, Daniel Moroz J, Rithvik Rao, and David Parkes C. 2021. Low-cost attacks on Ethereum 2.0 by sub-1/3 stakeholders. In *Proceedings of WINE 2020 Workshop on Game Theory in Blockchain*.
- [24] Yundi Qian, B William, and Milind Tambe. 2015. Robust Strategy against Unknown Risk-averse Attackers in Security Games. In *AAMAS*, Vol. 15. 1341–49.
- [25] Ayelet Sapirshstein, Yonatan Sompolskiy, and Aviv Zohar. 2016. Optimal selfish mining strategies in bitcoin. In *FC*.
- [26] Alistair Stewart. 2019. Poster: Grandpa finality gadget. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. 2649–2651.
- [27] Gavin Wood. 2016. Polkadot: Vision for a heterogeneous multi-chain framework. *White paper* 21, 2327 (2016), 4662.
- [28] Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. 2019. HotStuff: BFT consensus with linearity and responsiveness. In *PODC*.
- [29] Mingfei Zhang, Rujia Li, and Sisi Duan. 2024. Max attestation matters: making honest parties lose their incentives in ethereum PoS. In *Proceedings of the 33rd USENIX Conference on Security Symposium*. 6255–6272.
- [30] Mingfei Zhang, Rujia Li, Xueqian Lu, and Sisi Duan. 2025. Available Attestation: Towards a Reorg-Resilient Solution for Ethereum Proof-of-Stake. In *USENIX Security*.
- [31] Xinrui Zhang, Rujia Li, Qin Wang, Qi Wang, and Sisi Duan. 2023. Time-manipulation Attack: Breaking Fairness against Proof of Authority Aura. *WWW* (2023).

A Related Work

Selfish mining attacks. The concept of selfish mining was first introduced by Eyal and Sirer [14] in the context of Bitcoin's Proof-of-Work consensus. Their seminal work demonstrated that rational miners could deviate from the honest protocol to gain disproportionate rewards by withholding blocks and strategically releasing private chains. This fundamental insight challenged the assumption that following the protocol honestly was always the optimal strategy for miners.

Subsequent research extended selfish mining analysis to various PoW systems, with notable contributions by Sapirshstein et al. [25] who provided optimal strategies for different network conditions, and Nayak et al. [21] who introduced stubborn mining as a generalization of selfish strategies. These works established the

theoretical framework for understanding strategic mining behavior in longest-chain protocols.

The transition to Proof-of-Stake systems introduced new dimensions to selfish mining analysis. Brown-Cohen et al. [5] provided a foundational framework distinguishing between local and global predictability models in PoS protocols. Their work demonstrated that global predictability enables risk-free selfish mining, where adversaries can predict future leadership roles and guarantee attack success. However, they left open the question of whether risk-free attacks were possible in local predictability models.

Neuder et al. [22] analyzed the selfish mining on PoS protocol of Tezos blockchain. Their analysis of Tezos is seen as a real-world example of the predictable selfish mining attack that was theorized by Brown-Cohen et al. [5]. Although Tezos utilizes the longest chain rule to select the canonical chain, it does not use VRF-based leader election mechanism.

As mentioned in the introduction, our selfish mining attacks are risk-free attacks in the hybrid predictability model. Such a model has never been studied yet, to our knowledge.

Malicious reorganization attacks. As mentioned in previous sections, selfish mining can be viewed as one type of malicious reorganization attacks (reorg attack for short). Neuder et al. [23] demonstrated low-cost attacks on Ethereum 2.0 by sub-1/3 stakeholders, showing how adversaries below the traditional Byzantine threshold could still cause significant disruptions through strategic reorganizations. Li et al. [18] found that by controlling only 13.4% stake, a selfish mining attack is already profitable in Ethereum 2.0. Zhang et al. [29] exploited a variant of the reorg attack to make honest validators suffer from penalties in Ethereum 2.0. These studies revealed that the transition to PoS did not eliminate reorganization risks but rather transformed their nature. Zhang et al. [30] proposed a provably secure solution that can mitigate all known reorg attacks in Ethereum PoS. The main idea of this solution is to ensure that validators only vote for a chain with blocks seen by more than one-third of validators.

Time manipulation attacks represent another protocol flaw for causing chain reorganizations. Zhang et al. [31] demonstrated time-manipulation attacks breaking fairness against Proof of Authority Aura consensus, showing how adversaries could exploit timestamp-based mechanisms to gain unfair advantages and cause localized reorganizations.

B Some Known Selfish Mining Attacks

B.1 Selfish Mining in Bitcoin

Selfish mining in Bitcoin was first introduced by Eyal and Sirer [14]. This attack demonstrates that the Bitcoin mining protocol is not incentive-compatible, allowing colluding miners to obtain revenue larger than their fair share without controlling a majority of the hash power.

Attack Strategy. The core idea of selfish mining is straightforward: instead of immediately broadcasting newly mined blocks, the attacker maintains a private branch and strategically releases it to maximize rewards. The attack exploits the longest-chain rule used by Bitcoin's consensus protocol. When the private chain becomes longer than the public chain, honest miners will abandon their

blocks and switch to the attacker's chain, causing honest work to be wasted.

Attack Scenario. Consider the following scenario to illustrate how the attack works. Suppose the attacker mines a block B_1 extending the current blockchain. Instead of broadcasting B_1 immediately, the attacker withholds it and continues mining privately. Meanwhile, honest miners continue working on the public chain.

If an honest miner finds a competing block B'_1 at the same height, the attacker immediately releases B_1 to create a fork. With probability γ , some fraction of honest miners will receive B_1 first and mine on it. The attacker continues mining on their private branch. If the attacker mines the next block B_2 before honest miners, the attacker releases B_2 , making their chain longer. All honest miners then switch to the attacker's chain, and the honest block B'_1 (and any subsequent blocks built on it) are discarded. The attacker thus earns rewards for both B_1 and B_2 , while the honest miners lose their mining rewards.

If the attacker maintains a lead of two or more blocks in the private chain, they can continue withholding until honest miners nearly catch up, then release just enough blocks to maintain the lead. This strategy allows the attacker to consistently waste honest mining effort.

B.2 Forking Attack in Ethereum 2.0

Nagy et al. [20] introduced the RANDAO forking attack against Ethereum 2.0's Proof-of-Stake consensus mechanism. This attack differs fundamentally from Bitcoin's selfish mining as it exploits Ethereum's distributed randomness beacon (RANDAO) and the slot-based block proposal mechanism in Proof-of-Stake protocols.

Attack Strategy. In Ethereum 2.0, validators are selected to propose blocks based on randomness generated by RANDAO. The forking attack exploits the fact that validators can manipulate this randomness by selectively forking out honest proposers' blocks. Unlike traditional block withholding attacks, where validators simply hide blocks, the forking attack involves strategically creating an alternative chain that excludes certain honest blocks to influence future validator selections and maximize transaction fee revenues.

Attack Scenario. Consider the following scenario. Suppose in slot t , an honest validator V_h is selected as the block proposer and produces block B_h . The attacker observes that if B_h is included in the canonical chain, the RANDAO value will be updated in a way that is unfavorable to the attacker (e.g., the attacker will not be selected as a proposer in profitable future slots).

To manipulate the outcome, a malicious validator V_a (who also has proposer rights in slot t or controls sufficient attestation power) creates an alternative block B_a at the same slot t . The attacker then uses their controlled validators to attest to B_a instead of B_h . If the attacker controls a sufficient stake (e.g., 33%), their attestations can make B_a the canonical block, causing B_h to be forked out.

Critically, by excluding B_h from the chain, the RANDAO value evolves differently than it would have if B_h were included. This altered RANDAO value affects future proposer selections. If the manipulation results in the attacker being selected for more profitable slots in the near future, the attacker gains additional transaction fee revenues and block rewards that compensate for the cost of forking.

C MDP Formulation for Risk-Free Selfish Mining via Withholding

In this section, we provide the detailed Markov Decision Process (MDP) formulation for analyzing the expected revenue of our risk-free selfish mining attack via withholding. Formally, a MDP is defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$, where $\mathcal{S}$ is the set of all possible states, $\mathcal{A}$ is the set of actions available, $\mathcal{P}$ is the transition function, and $\mathcal{R}$ is the reward function.

State space. We define the state space of the MDP as $\mathcal{S} = \{(a, h) \mid a, h \in \mathbb{N}\}$, where:

- a represents the number of primary blocks in the private branch.
- h represents the number of primary blocks in the public branch.

The state $(0, 0)$ represents the initial state where no attack has been launched. Byzantine validators transition to state $(1, 0)$ when they control a primary leader and decide to start withholding blocks. The attack continues as long as Byzantine validators choose to withhold their private branch.

Action space. At each state (a, h) , Byzantine validators can choose from the action set $\mathcal{A} = \{(\text{continue}, \text{release})\}$:

- *continue*: Continue to withhold the private branch and wait for the next slot.
- *release*: Release all blocks in the private branch to all validators.

For our risk-free attack strategy, we simplify the attack strategy to fit MDP as: Byzantine validators release the private branch when $a - h = 1$. This ensures that the private branch always has at least as many primary blocks as the public branch at the moment of release. According to the fork choice rule (Section 2.2), when two chains have the same number of primary blocks, the first received chain becomes the canonical chain. Since Byzantine validators release their blocks before the next slot begins, the private branch will be received earlier and thus becomes the canonical chain.

State transition probabilities. When Byzantine validators choose to *continue* at state (a, h) , the state transitions to the next state based on the leader selection in the next slot. We denote the transition probabilities $\mathcal{P}$ as follows:

- With probability $p_a = \frac{f}{n} \cdot c$, a Byzantine validator is selected as a primary leader. The state transitions to $(a + 1, h)$.
- With probability $p_h = (1 - \frac{f}{n}) \cdot c$, an honest validator is selected as a primary leader. The state transitions to $(a, h + 1)$.
- With probability $p_0 = 1 - c$, no validator is selected as a primary leader. The state remains at (a, h) .

Here, f is the number of Byzantine validators, n is the total number of validators, and $c = 1/4$ is the BABE constant representing the probability that at least one primary leader exists in a slot.

When Byzantine validators choose to *release* at state (a, h) with $a \geq h$, the attack terminates.

Reward function. The reward function $\mathcal{R}$ depends on the action taken by Byzantine validators:

- If Byzantine validators choose *continue* at state (a, h) , they receive no immediate reward. The reward will be determined when the attack terminates.
- If Byzantine validators choose *release* at state (a, h) with $a \geq h$, the private branch becomes the canonical chain and the public

branch is discarded. Here, we consider the reward as the number of honest primary blocks h that are reorganized out of the canonical chain: $\mathcal{R} = h$

Note that in our risk-free attack, Byzantine validators never release when $a < h$, so they never lose any revenue.

Expected Revenue Analysis. To compute the expected additional revenue, we need to analyze the steady-state behavior of the MDP under the threshold policy where Byzantine validators release when $a - h = 1$.

Let $\pi(a, h)$ denote the steady-state probability of being in state (a, h) . The expected additional reward per attack instance can be computed as:

$$\mathbb{E}[\mathcal{R}] = \sum_{h=0}^d \pi(h+1, h) \cdot h \quad (1)$$

This represents the sum over all states where Byzantine validators release their private branch (i.e., states of the form $(h+1, h)$), weighted by the steady-state probability of reaching each state and the reward obtained h .

To compute the steady-state probabilities, we solve the balance equations for the Markov chain. Starting from state $(2, 0)$ when the attack begins, the state evolves according to the transition probabilities defined earlier. The key observation is that the attack terminates whenever $a - h = 1$, at which point Byzantine validators release and collect h additional reward.

To compute the expected additional revenue, we use Monte Carlo simulation to estimate the steady-state behavior of the attack. For each attack instance, we start from state $(2, 0)$ and simulate the state evolution according to the transition probabilities p_a , p_h , and p_0 . The simulation continues until the release condition $a - h = 1$ is satisfied. We record the value of h at the time of release, which represents the number of honest primary blocks that are reorganized out of the canonical chain. We repeat this simulation for 100,000 attack instances and compute the average value of h , denoted as $\mathbb{E}[h]$. Our simulation results for various values of f/n with $c = 1/4$ are shown in Table 3.

Byzantine fraction (f/n)	additional reward (%)
0.05	0.04
0.10	0.20
0.15	0.45
0.20	0.75
0.25	1.16
0.33	2.00

Table 3: Monte Carlo simulation results for the withholding attack with 100,000 simulation runs per configuration.

D Empirical Measurement on Polkadot Mainnet

To validate the practical feasibility of our proposed attacks, we conducted empirical measurements on the Polkadot mainnet from June to September 2025.

We use the Polkadot official JavaScript API to capture real-time block propagation data from the mainnet. Our monitoring script established a WebSocket connection to the official Polkadot RPC

endpoint ([wss://rpc.polkadot.io](https://rpc.polkadot.io)) and subscribed to new block headers via the `api.rpc.chain.subscribeNewHeads` method. For each received block, we recorded the precise timestamp of reception using `Date().toISOString()` and extracted the block number, slot number, and block type (primary or secondary) from the header data.

We define the *block propagation time* as the duration between the start of a slot and the moment our monitoring node receives the corresponding block. Formally, for a block b proposed in slot t , the propagation time is calculated as:

$$\text{PropagationTime}(b) = T_{\text{received}} - (t \times 6 \text{ seconds})$$

where T_{received} is the wall-clock timestamp when our node received block b , and each slot lasts 6 seconds in Polkadot. This metric captures the end-to-end delay from block production to network-wide dissemination.

Our monitoring system collected data continuously over the three-month period, capturing the data of 300,000 blocks, consisting of 75,599 primary blocks and 224,401 secondary blocks.

Primary block ratio analysis. Our measurements show that primary blocks constitute 25.2% of all observed blocks (75,599 out of 300,000), very close to the BABE constant $c = 0.25$. A one-sample proportion z-test for $H_0: p = 0.25$ yields $z = 2.53$ (two-tailed $p \approx 0.011$). The 95% Wilson confidence interval is $[0.25045, 0.25353]$, which excludes 0.25 by a very small margin (absolute difference 0.002, relative difference 0.8%).

Block propagation time analysis. As shown in Fig. 9, the mean propagation time is 0.692 seconds, with a median of 0.610 seconds, indicating that the majority of blocks are received within the first second after the corresponding slot begins. The first and third quartiles ($Q1 = 0.526s$, $Q3 = 0.792s$) show that 50% of all blocks arrive within a tight window of approximately 0.27 seconds. Comparing the propagation between primary and secondary blocks reveals similar distributions. Both block types exhibit identical median values and comparable interquartile ranges.

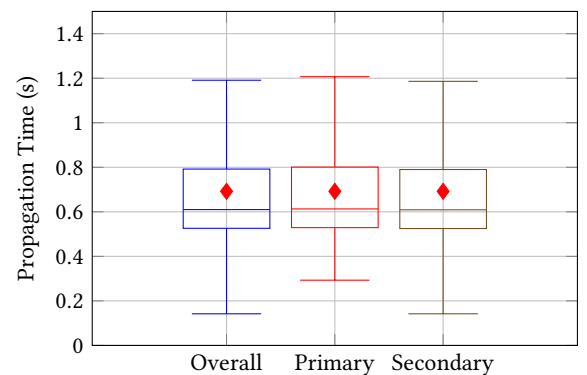


Figure 9: Block propagation time comparison across different block types. (a) Overall distribution. (b) Primary blocks. (c) Secondary blocks. The boxes show the interquartile range (Q1-Q3), the horizontal line indicates the median, and the diamond marker shows the mean.

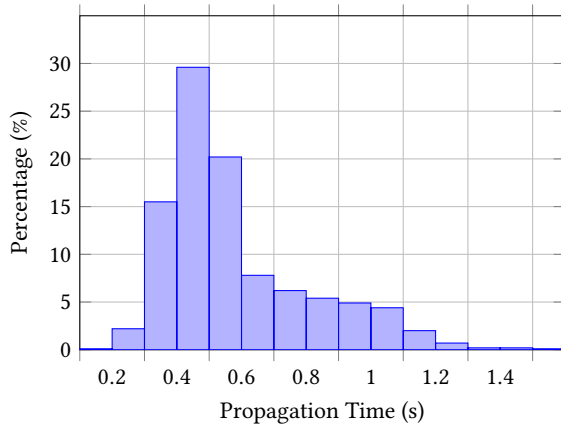


Figure 10: Histogram of block propagation times for all blocks in the dataset.

Figure 10 presents the distribution of propagation times across all blocks. The histogram shows a right-skewed distribution with a peak around 0.6 seconds, confirming that the vast majority of blocks are received shortly after the slot begins. Notably, the distribution is highly concentrated: 87.1% of all blocks arrive within the first

second of their respective slot periods, and 99.7% arrive within two seconds.

Implications for attack feasibility. For the attack via withholding (Section 3), the fork choice rule breaks ties between chains of equal length by selecting the first-received chain. Our measurements demonstrate that Byzantine validators maintain a decisive advantage in this race condition. When the termination condition is satisfied (i.e., $|C_{t,t'}^{\text{adv}}| = |C_{t,t+j}^{\text{hon}}|$), Byzantine validators can immediately broadcast their pre-constructed private chain. Since these blocks have already been created and are ready for instant dissemination, they propagate through the network with minimal delay.

In contrast, honest validators in the competing public chain must first propose blocks, which then undergo the typical 0.6-0.8 second propagation process. Even in the worst case, where an honest primary block is proposed immediately after the slot begins, Byzantine blocks released simultaneously will reach most validators earlier. The median propagation time of 0.61 seconds measured in our study represents the time from slot start to block reception, but Byzantine validators eliminate this delay by pre-releasing completed blocks. This timing asymmetry ensures that in competitions where $|C_{t,t'}^{\text{adv}}| = |C_{t,t+j}^{\text{hon}}|$, the private chain will be adopted as canonical with high probability, making the withholding attack risk-free in practice.